Neuron-Dropout Faults in Fractional-Order Reservoirs: A Theoretical Analysis

Navid Shamszadeh

4 October 2026

Abstract

Clamping a neuron's weights to zero acts on a fractional-order reservoir (FOR) as an injected "anti-signal": the exact negative of what that neuron used to send. The reservoir's response is that forcing passed through the fractional dynamics. In a linearized model this yields closed-form results with five consequences.

- **The path depends on α.** After a permanent fault the reservoir relaxes to its new state as a power law, $t^{-\alpha}$, not exponentially. With an untruncated Grünwald–Letnikov (GL) history there is no characteristic recovery time.
- **The endpoint does not, at a fixed operating point.** Given the neurons' mean rates and gains, the fault's shift of the mean operating point is independent of α. With a truncated GL buffer, α and L enter through an effective leak. In a spiking reservoir the rates and gains themselves depend on α, so the endpoint does too, but only through them.
- **In hardware, the buffer length matters as much as α.** Truncation at L steps caps recovery: for $\tau_m = 10$ steps at about $1.5L$ to $2.5L$ steps, and longer for slower membranes. For short buffers, recovery time peaks at intermediate α.
- **Lost memory capacity with a retrained readout is exactly 1 per neuron in the linear, noise-free limit.** That holds for every α. Any α-dependence of fault tolerance measured with retraining therefore comes from noise, spiking nonlinearity or quantization, which tells you where to look.
- **Random dropout moves the reservoir toward the ordered regime.** The effective coupling gain shrinks by a factor of $\sqrt{1-p}$ when a fraction p of neurons is removed.

The derivations use the GL-FLIF model and the information measures of Mastin et al. [1]. A numerical check of the main scaling laws is included.

1 Model and fault definitions

The analysis linearizes the spiking GL-FLIF reservoir of Mastin et al. [1] into a linear fractional-order network. That model is exactly solvable, and its predictions are then checked against the spiking model.

1.1 The GL-FLIF reservoir

Each neuron i follows Algorithm 1 of [1], with Grünwald–Letnikov coefficients $c_k = (-1)^k \binom{\alpha}{k}$ and a history buffer of length L:

$$V_{i,n} = \Delta t^{\alpha} \left(-\frac{V_{i,n-1} - V_{\text{rest}}}{\tau_m} + I_{i,n} + b \right) - \sum_{k=1}^{L} c_k \, V_{i,n-k}, \qquad I_{i,n} = \sum_j W_{ij} \, s_{j,n-1} + \sum_m W^{\text{in}}_{im} \, u_{m,n}, \tag{1}$$

with threshold V_{th}, reset to V_{reset}, spikes s_j and a linear readout $y_n = W_{\text{out}} x_n$. For $0 < \alpha < 1$, every c_k with $k \geq 1$ is negative and $\sum_{k\geq 0} c_k = 0$. So $-\sum_k c_k V_{n-k}$ is a weighted average of past potentials whose weights $|c_k|$ decay as $k^{-1-\alpha}$. For $\alpha = 1$ only $c_1 = -1$ survives and the update reduces to the forward-Euler LIF neuron.

Lemma 1. *For $0 < \alpha < 1$ and $k \geq 1$ every $c_k(\alpha)$ is negative and*

$$\sum_{k=0}^{\infty} c_k(\alpha) = 0$$

Proof. We first show the ratio

$$\frac{c_k}{c_{k-1}} = \frac{k-1-\alpha}{k}$$

To see this we expand c_k and c_{k-1}:

$$\frac{c_k}{c_{k-1}} = \frac{(-1)^k \binom{\alpha}{k}}{(-1)^{k-1}\binom{\alpha}{k-1}} = -\frac{\binom{\alpha}{k}}{\binom{\alpha}{k-1}}$$

Note that for positive non-integer α we use the Gamma function $\Gamma(\cdot)$ for binomial coefficients:

$$\binom{\alpha}{k} = \frac{\Gamma(\alpha+1)}{\Gamma(k+1)\Gamma(\alpha-k+1)} \implies \frac{\binom{\alpha}{k}}{\binom{\alpha}{k-1}} = \frac{\frac{\Gamma(\alpha+1)}{\Gamma(k+1)\Gamma(\alpha-k+1)}}{\frac{\Gamma(\alpha+1)}{\Gamma(k)\Gamma(\alpha-k+2)}}$$

$$= \frac{\Gamma(k)\Gamma(\alpha-k+2)}{\Gamma(k+1)\Gamma(\alpha-k+1)}$$

Note that $\Gamma(z+1) = z\Gamma(z)$ for any complex number with positive real part. Therefore we can simplify in the following manner:

$$\frac{(\alpha-k+1)\Gamma(k)\Gamma(\alpha-k+1)}{k\Gamma(k)\Gamma(\alpha-k+1)} =$$

□

1.2 Linearization

The linear theory needs each neuron's spike output written as a smooth function of a continuous state. The membrane potential itself will not do: it is reset after every spike, so its trajectory is a sawtooth and spiking is not a smooth function of it. The state used instead is the *free membrane potential* V^f, the potential the neuron would have with threshold and reset removed. It is neuron i's input passed through the fractional membrane filter:

$$V^f_{i,n} = \Delta t^{\alpha} \left(-\frac{V^f_{i,n-1} - V_{\text{rest}}}{\tau_m} + I_{i,n} + b \right) - \sum_{k=1}^{L} c_k \, V^f_{i,n-k}. \tag{2}$$

Around a stationary operating point, each neuron's spike output is replaced by its linear response in the free potential:

$$s_{j,n} \approx r_j + \gamma_j \left(V^f_{j,n} - \bar{V}_j \right). \tag{3}$$

- r_j**, the mean rate**, in spikes per step. Since $s_{j,n} \in \{0,1\}$, it is the probability that neuron j spikes in a given step. It is measured from the golden run as the time average of $s_{j,n}$ after the initial transient, or computed from the mean-field condition $r_j = f_j(\bar{I}_j)$, with $\bar{I}_j = \sum_k W_{jk} r_k + \sum_m W^{\text{in}}_{jm} \bar{u}_m$ and f_j the neuron's f–I curve (below).

- $\bar{V}_j$**, the mean free potential** at the operating point: $\bar{V}_j = G_{\mathrm{DC}}\left(V_{\mathrm{rest}}/\tau_m + \bar{I}_j + b\right)$, where $G_{\mathrm{DC}} = (1/\tau_m + \varepsilon_L/\Delta t^{\alpha})^{-1}$ is the DC gain of the membrane filter and ε_L is the truncation leak of Eq. (12). For an untruncated buffer, $G_{\mathrm{DC}} = \tau_m$ and $\bar{V}_j = V_{\mathrm{rest}} + \tau_m(\bar{I}_j + b)$. It is not the mean of the actual, reset-sawtooth potential.
- γ_j**, the gain per unit free potential**: $\gamma_j = f_j'(\bar{I}_j)/G_{\mathrm{DC}}$. The f–I curve $f_j(I)$ gives the firing rate (f, for frequency) as a function of mean input I, and its slope f_j' at the operating point is the f–I gain. Dividing by G_{DC} makes the linear model reproduce the f–I curve at DC, since a change ΔI in mean input moves the free potential by $G_{\mathrm{DC}}\,\Delta I$.

The f–I curve here is the *noise-averaged* one: rate against mean input, with input fluctuations held at the variance neuron j sees in the golden run. The reservoir is driven, and those fluctuations smooth the curve near the rheobase, where the deterministic curve's slope diverges. The curve depends on α (Mastin et al. [1] show that lower α raises the effective rheobase), on L and on the fixed-point format, so γ_j does too.

There are two practical ways to get r_j, γ_j and $\bar{V}_j$. One is to drive a single GL-FLIF neuron with input matching neuron j's golden-run statistics, sweep the mean and take the slope of the measured rate. The other uses the golden run directly: replay the recorded $I_{j,n}$ through Eq. (2), then regress $s_{j,n}$ on $V^f_{j,n}$. The slope estimates γ_j and the mean of $V^f_{j,n}$ gives $\bar{V}_j$. The regression includes truncation and quantization automatically and costs nothing extra on the HiL rig.

The linear FOR follows from Eq. (2) by substituting the linear spike response into $I_{i,n}$. Neuron i receives the previous step's spikes, so Eq. (3) is applied at $n-1$, and the recurrent input splits into a part that depends on the state and a constant:

$$\sum_j W_{ij}\, s_{j,n-1} \approx \sum_j W_{ij}\,\gamma_j\, V^f_{j,n-1} + \sum_j W_{ij}\left(r_j - \gamma_j \bar{V}_j\right).$$

This is the only approximation in the linear model. The coefficient $W_{ij}\gamma_j$ is the (i,j) entry of $\tilde{W} = W\,\mathrm{diag}(\gamma)$: each synapse is scaled by the gain of the neuron that sends it. The leak splits the same way, $-(V^f_{i,n-1} - V_{\mathrm{rest}})/\tau_m = -V^f_{i,n-1}/\tau_m + V_{\mathrm{rest}}/\tau_m$.

Now take the free potentials as state, $x_n = V^f_n$, and sort the terms inside the parentheses of Eq. (2) into three groups:

- **Previous state:** $-x_{n-1}/\tau_m + \tilde{W}x_{n-1} = A\,x_{n-1}$. The leak and the recurrent coupling share one matrix because both act on x_{n-1}: the leak because Eq. (2) evaluates it at $n-1$, the coupling because of the one-step spike delay.
- **Input:** $W^{\mathrm{in}}u_n = B\,u_n$, which enters at step n with no delay.
- **Constants:** $(V_{\mathrm{rest}}/\tau_m + b)\mathbf{1} + W\left(r - \mathrm{diag}(\gamma)\,\bar{V}\right) = \beta$, where $\mathbf{1}$ is the all-ones vector.

The GL coefficients c_k are scalars shared by all neurons, so the history sum carries over unchanged. No threshold remains: the state never resets, and the other neurons' spikes enter only through their linear response. Taking the approximation as an equality defines the *linear FOR*:

$$\begin{gathered} x_n = \Delta t^{\alpha}\left(A\,x_{n-1} + B\,u_n + \beta\right) - \sum_{k=1}^{L} c_k\, x_{n-k}, \\ A = -\tfrac{1}{\tau_m}\mathrm{Id} + \tilde{W}, \quad \tilde{W} = W\,\mathrm{diag}(\gamma), \quad B = W^{\mathrm{in}}, \quad \beta = \left(\tfrac{V_{\mathrm{rest}}}{\tau_m} + b\right)\mathbf{1} + W\left(r - \mathrm{diag}(\gamma)\,\bar{V}\right). \end{gathered} \tag{4}$$

In the continuous limit ($\Delta t \to 0$ with the memory window $L\Delta t \to \infty$) this is $D^{\alpha}x = Ax + Bu + \beta$. A neuron far below threshold has $r_j = 0$ and $\gamma_j = 0$ and drops out of $\tilde{W}$, so removing it is harmless except through the readout. A silent neuron near threshold is different: under the noise-averaged curve its γ_j is small but nonzero, and a fault can switch it on (Section 6.1).

The linearization has two known gaps. A static gain on the free potential describes the spike response well at frequencies below the firing rate, but it misses the reset and refractory effects

that shape the response at higher frequencies, so frequency-domain results such as $H'(\omega)$ are approximate at the high end. And the fluctuations of the binary spike train around $r_j+\gamma_j(V_j^f-\bar{V}_j)$ are dropped. They act as state noise, which matters in Section 5.

1.3 Three variants of "set its weights to zero"

"All of its edge weights" can mean incoming, outgoing or both, and the variants behave differently (Table 1). The analysis treats full removal as the primary fault and decomposes it into the other two.

Variant	What is zeroed	What neuron j does afterwards	Error channels
F-out (silenced)	Column j of W (outgoing synapses)	Still driven by input and still read out	Propagated only
F-in (deafened)	Row j of W and of W_{in} (incoming)	Relaxes to a bias-driven state that carries no input information; may fire periodically if b exceeds the rheobase	Direct and propagated
F-all (removed)	Row and column j, row j of W_{in}, readout weight w_j	Gone	Direct and propagated

Table 1: Three variants of the zero-weights fault.

The *direct channel* is what the readout loses by no longer seeing neuron j. The *propagated channel* is the change in every other neuron's state because neuron j stopped talking to them.

2 The fault as a forcing term

The fault's whole effect on the surviving neurons is the response of the faulty network to the negative of neuron j's former output. Because the nominal and faulty runs share their entire history up to the fault time t_f, that response starts from zero history, which removes the usual difficulty with initial conditions in fractional systems.

2.1 Perturbation equation

Let x be the nominal trajectory and x' the faulty one, and let $\delta = x' - x$ on the surviving neurons. Write A' for A with row and column j deleted, q_j for column j of W restricted to the survivors, and $a_j = \gamma_j q_j$ for the corresponding column of $\tilde{W}$. Neuron j sends $q_j s_j$, and in the linear FOR that signal is split between the state term $a_j x_j$ and the constant $q_j(r_j - \gamma_j \bar{V}_j)$ inside β. Removing the neuron removes both. Subtracting the two linear FOR equations gives, for F-all:

$$D^\alpha \delta(t) = A'\,\delta(t) - q_j\, s_j(t), \qquad s_j(t) = r_j + \gamma_j\, \tilde{x}_j(t), \qquad \delta(t) = 0 \;\text{ for } t < t_f, \tag{5}$$

where $\tilde{x}_j = x_j - \bar{V}_j$ is the fluctuation of neuron j's free potential about its mean. The forcing $-q_j s_j(t)$ is exactly what neuron j used to send, with its sign flipped. Its mean is $-q_j r_j$, set by the mean firing rate, not $-a_j \bar{V}_j$; the two agree only if the linearized spike response passes through the origin, $r_j = \gamma_j \bar{V}_j$. For F-out the equation is the same and neuron j keeps its readout term. For F-in, $s_j(t)$ is replaced by the difference between its nominal output and the output of the deafened neuron. The deafened neuron settles at the bias-driven rate $f_j(b)$, usually far from the original operating point, so that output comes from the f–I curve rather than from the linearization.

2.2 Solution

In continuous time the solution is a convolution with the fractional impulse response, written with the two-parameter Mittag-Leffler function $E_{\alpha,\beta}$ [8]:

$$\delta(t) = -\int_{t_f}^{t} G(t-s)\, q_j\, s_j(s)\, ds, \qquad G(t) = t^{\alpha-1} E_{\alpha,\alpha}\big(A' t^{\alpha}\big)\,. \tag{6}$$

For $\alpha = 1$, $G(t) = e^{A't}$ and this reduces to the familiar exponential convolution. Numerically, $E_{\alpha,\beta}$ can be evaluated with the method of Garrappa [9].

The discrete GL scheme that runs on the FPGA has an exact z-domain counterpart. Per eigenmode λ of A', the transfer function from forcing to state is

$$H_\lambda(z) = \frac{\Delta t^{\alpha}}{C_L(z) - \Delta t^{\alpha}\,\lambda\, z^{-1}}, \qquad C_L(z) = \sum_{k=0}^{L} c_k z^{-k} \xrightarrow{L\to\infty} (1-z^{-1})^{\alpha}. \tag{7}$$

The branch point of $(1-z^{-1})^{\alpha}$ at $z=1$ is what produces power-law tails in time; truncating to a polynomial C_L removes it.

2.3 Two channels and two components

With the readout held fixed, the output error after an F-all fault is

$$e(t) = y'(t) - y(t) = \underbrace{-\,w_j\, x_j(t)}_{\text{direct}} + \underbrace{\textstyle\sum_{i\neq j} w_i\, \delta_i(t)}_{\text{propagated}}. \tag{8}$$

The two channels can partly cancel, so the error variance includes a cross term that can be negative.

It also helps to split neuron j's signal into its mean and its fluctuation. The readout sees the free potential, $x_j = \bar{x}_j + \tilde{x}_j$ with $\bar{x}_j = \bar{V}_j$; the survivors see the spike output, $s_j = r_j + \gamma_j \tilde{x}_j$:

- **The mean** acts as a step input at t_f. Through the propagated channel, the lost mean drive $q_j r_j$ shifts the operating point of every downstream neuron; Sections 3 and 4 describe how and where to. Through the direct channel, the readout loses the constant $w_j \bar{x}_j$.
- **The fluctuation** carries input information. Its removal, $\gamma_j q_j \tilde{x}_j = a_j \tilde{x}_j$ in the propagated channel and $w_j \tilde{x}_j$ in the direct one, acts like injecting structured, input-correlated noise. This component determines the information loss in Section 5.

3 Recovery dynamics

A permanent fault relaxes toward its new state as $t^{-\alpha}$, and a transient fault decays as $t^{-1-\alpha}$. Both are power laws with no characteristic time. On real hardware the GL buffer length L caps them: after about L steps the decay turns exponential, with a time constant set by L, α and τ_m (Section 3.3).

3.1 Permanent fault: step forcing

For one eigenmode λ of A' and a constant forcing F (the projection onto that mode of $-q_j r_j$, the negative of the mean drive neuron j used to send), the response is the fractional step response [8]:

$$\delta(t) = \delta^*\left[1 - E_\alpha(\lambda(t-t_f)^{\alpha})\right], \qquad \delta^* = -F/\lambda, \qquad E_\alpha(\lambda t^{\alpha}) \approx \frac{-1}{\Gamma(1-\alpha)\,\lambda\, t^{\alpha}} \quad (t\to\infty). \tag{9}$$

Setting the relative distance to the new state equal to ε gives the ε-recovery time. For $\alpha < 1$ it scales as a power of $1/\varepsilon$, not as its logarithm:

$$T_\varepsilon(\alpha < 1) \approx (\varepsilon\,|\lambda|\,\Gamma(1-\alpha))^{-1/\alpha}, \qquad T_\varepsilon(\alpha = 1) = |\lambda|^{-1}\ln(1/\varepsilon). \tag{10}$$

For $\tau_m = 10$ steps ($|\lambda| = 0.1$) and $\varepsilon = 1\%$, the integer-order neuron recovers in about 46 steps. An untruncated $\alpha = 0.5$ neuron would need about 318,000. The accumulated excess error $\int |\delta - \delta^*|\,dt$ diverges for every $\alpha < 1$, because $t^{-\alpha}$ is not integrable.

3.2 Transient fault: pulse forcing

If neuron j is removed for a short duration d and then restored (a transient fault, or a crude stand-in for an upset), the forcing is a pulse and the response is the impulse response G:

$$\delta(t) \approx F d\, G(t - t_f), \qquad G(t) \sim \frac{t^{-1-\alpha}}{\lambda^2\,|\Gamma(-\alpha)|} \quad (t \to \infty). \tag{11}$$

Here F is the same mode forcing as in Eq. (9). Since $G > 0$, the tail has the sign of the permanent shift $\delta^* = -F/\lambda$. This tail is integrable, so a transient fault costs finite cumulative error. The cost is still heavy-tailed compared with the exponential $e^{\lambda t}$ of an integer-order neuron.

3.3 What truncation changes

The FPGA keeps only L past samples. Truncation leaves a positive residual in the coefficient sum, which acts as an extra leak per step:

$$\varepsilon_L = \sum_{k=0}^{L} c_k = \frac{\Gamma(L+1-\alpha)}{\Gamma(1-\alpha)\,\Gamma(L+1)} \approx \frac{L^{-\alpha}}{\Gamma(1-\alpha)}, \qquad \lambda \;\to\; \lambda - \varepsilon_L/\Delta t^\alpha \quad \text{(at DC)}. \tag{12}$$

So the power law holds for $n \ll L$, after which decay turns exponential. For $\alpha = 0.5$ and $L = 64$, $\varepsilon_L \approx 0.07$, comparable to the 0.1 leak of $\tau_m = 10$ steps. The predicted steady-state gain drops from 10 to 5.87, and simulation of Algorithm 1 gives 5.87.

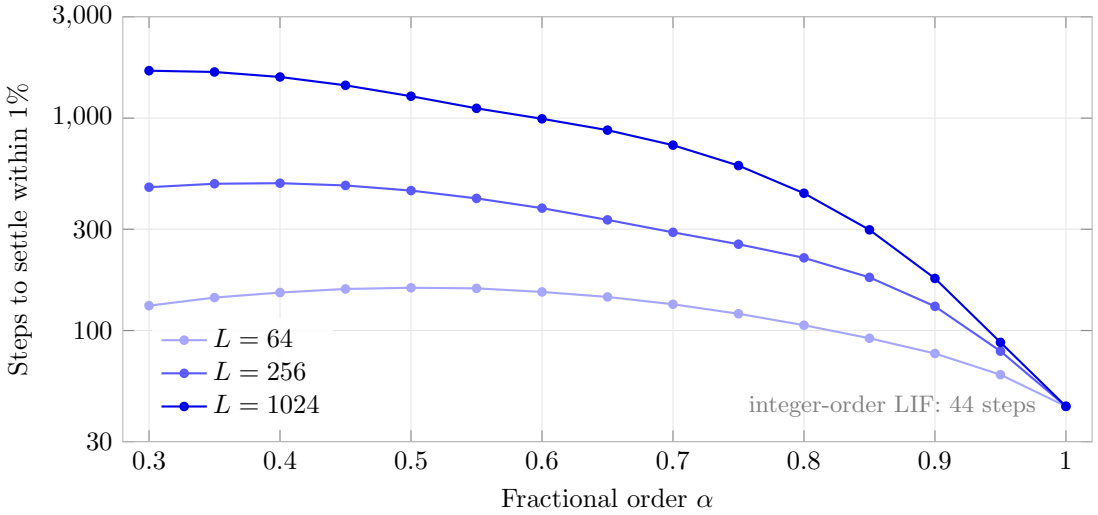


Figure 1: For $\tau_m = 10$ steps a fault lingers for at most about $2.5L$ steps, and short buffers peak at intermediate α. Steps to settle within 1% of the new state after a permanent step-like fault, by GL buffer length L. Simulated scalar GL-FLIF step response (Algorithm 1 of [1] without threshold), $\Delta t = 1$, $\tau_m = 10$ steps.

With a long buffer ($L = 1024$), lower α always means slower recovery (Figure 1). With a short buffer ($L = 64$), the truncation leak dominates at low α, so recovery time peaks near $\alpha \approx 0.5$. In every case it stays below about $2.5L$ steps, against the untruncated 318,000 at $\alpha = 0.5$.

The ratio to L is specific to $\tau_m = 10$ steps. Once the buffer is truncated, the slowest decay of a real mode $\lambda < 0$ is set by the pole of $H_\lambda(z)$ (Eq. (7)) on the real axis closest to $z = 1$, that is, the real root z^* of $C_L(z) = \Delta t^\alpha \lambda z^{-1}$, which gives a time constant of $-1/\ln z^*$ steps. The 1% recovery time is then at most about

$$T_{1\%} \lesssim \min\left(T_\varepsilon,\ \frac{\ln 100}{|\ln z^*|}\right), \tag{13}$$

with T_ε from Eq. (10) at $\varepsilon = 1\%$. In simulations over $\tau_m \in \{10, 100\}$ steps, $L \in \{64, 256, 1024\}$ and α from 0.3 to 1, this estimate never fell below the simulated recovery time and overestimated it by at most a factor of 1.7. The cap grows with τ_m. At $\alpha = 1$ it is $\tau_m \ln 100 \approx 4.6\,\tau_m$ whatever L is, and for $\tau_m = 100$ steps and $L = 64$ recovery takes up to $8.4L$ (537 steps at $\alpha = 0.9$). The figure's cap of about $2.5L$ therefore does not transfer to other membrane time constants; for the hardware, evaluate Eq. (13) at its own τ_m.

3.4 Check in a network

The exponents were checked in a linear GL network of 100 neurons with random Gaussian coupling ($g\tau_m = 0.8$), $\tau_m = 2$ steps, $L = 6000$, and an F-all fault at step 3000. The transient fault removed the neuron for 20 steps. For the permanent fault, the slope of $d\delta/dt$ was fitted, which should scale as $t^{-1-\alpha}$. Fits used 300 to 2500 steps after the fault (Table 2).

α	Predicted exponent	Permanent fault, fitted	Transient fault, fitted
0.4	-1.4	-0.95	-1.20
0.6	-1.6	-1.54	-1.63
0.8	-1.8	-1.81	-1.90

Table 2: Predicted and fitted power-law exponents after a single-neuron fault.

The $\alpha = 0.4$ mismatch is expected: the asymptotic form needs $|\lambda| t^\alpha \gg 1$, that is $t \gg |\lambda|^{-1/\alpha} \approx 316$ steps for the slowest mode, which is where the fit window begins. Low-α experiments will need long observation windows to see the predicted tail.

4 Where the reservoir settles

At a fixed operating point, the fault's shift of the mean operating point does not depend on α, but its effect on how the reservoir responds to inputs does. Removing a random neuron also slightly shrinks the reservoir's spectrum, pushing it toward the ordered regime.

4.1 The mean shift is independent of α

The fractional derivative of a constant is zero (in the GL form because $\sum_k c_k = 0$), so every α has the same fixed point. For the constant part of the forcing in Eq. (5), the new steady state of the survivors is

$$\delta^* = A'^{-1} q_j\, r_j \quad (L \to \infty), \qquad \delta_L^* = \left(A' - \tfrac{\varepsilon_L}{\Delta t^\alpha}\mathrm{Id}\right)^{-1} q_j\, r_j \quad \text{(truncated)}. \tag{14}$$

In words: given the operating point, α changes the path to the new operating point, not the point itself, unless the buffer is truncated. On the FPGA, α and L reach the endpoint through the effective leak ε_L.

The operating point itself is a different matter. The rates r and gains γ, and therefore A', come from the noise-averaged f–I curve, which depends on α (lower α raises the effective rheobase [1]). In a spiking reservoir the measured δ^* therefore changes with α through r_j and A', even though Eq. (14) has no explicit α. The α-independence is a statement about Eq. (14) at fixed (r, γ), and that is how it should be tested.

The fluctuations are different. The surviving reservoir's frequency response is $H'(\omega) = ((i\omega)^\alpha \mathrm{Id} - A')^{-1}$, which depends on α at every frequency except zero. So the information-carrying part of the fault's effect is α-dependent even though the mean shift is not.

4.2 Spectral effect of removing a neuron

For a specific neuron, first-order perturbation theory gives the shift of each eigenvalue λ_i, using its left and right eigenvectors v_i and u_i and the change ΔA from deleting row and column j:

$$\delta\lambda_i \approx \frac{v_i^* \, \Delta A \, u_i}{v_i^* u_i}, \qquad |\delta\lambda_i| \le \kappa(U) \, \|\Delta A\| \le \kappa(U) \left(\|\tilde{W}_{:,j}\| + \|\tilde{W}_{j,:}\| \right). \tag{15}$$

The bound is Bauer–Fike [11], with $\kappa(U)$ the condition number of the eigenvector matrix. Neurons with large incoming and outgoing weight norms, and large components in the slowest modes, are the ones whose removal moves the spectrum most.

For a random reservoir the average effect is simple. If $\tilde{W}$ has i.i.d. entries of variance g^2/N, its eigenvalues fill a disk of radius g (the circular law [12]). Deleting k neurons leaves an $(N-k)$-square matrix with the same entry variance, so

$$g_{\text{eff}} = g\sqrt{1-p}, \qquad p = k/N. \tag{16}$$

Here N, k and p count active neurons only. A neuron far below threshold has $\gamma_j = 0$, so its column of $\tilde{W}$ is zero and it does not contribute to the spectrum; removing it leaves g unchanged. If γ varies across neurons, the columns of $\tilde{W}$ have different variances and the radius is set by their average [13]: $g^2 = N_{\text{act}} \, \sigma_W^2 \langle \gamma_j^2 \rangle$, with σ_W^2 the entry variance of W and the average taken over the N_{act} active neurons.

Random dropout therefore makes the reservoir more stable and more strongly damped. The linearized reservoir loses stability when its eigenvalue disk reaches the leak, at $g\tau_m = 1$. In random rate networks this is also where chaos begins [3], and the transition persists with excitatory–inhibitory structure [4]. Input drive moves the onset of chaos to larger gain [5], so in a driven reservoir the edge of chaos lies somewhat above $g\tau_m = 1$. Either way, losing neurons lowers $g\tau_m$ and moves the reservoir into the ordered regime. With noise, memory capacity should fall too, since driven random networks compute best near this boundary [6]; in the noise-free linear model it does not (Section 5). A single removal changes g by only about $g/(2N)$, so this matters for cumulative faults, not single ones.

4.3 Stability

The continuous linear FOR is stable when every eigenvalue satisfies $|\arg\lambda| > \alpha\pi/2$ (Matignon's criterion [7]). Smaller α widens the stable sector into the right half-plane, so a fault that pushes a complex eigenvalue pair rightward can leave a low-α reservoir stable when an integer-order one would not be.

This protection does not extend to real eigenvalues. A real eigenvalue crossing zero destabilizes every α, and for circular-law reservoirs the first crossing is on the real axis at $g\tau_m = 1$. So α does not move the critical gain of a random reservoir; it only adds margin against faults that act on oscillatory modes.

Near criticality α does amplify the fault's effect on time scales. A mode near zero relaxes on the time scale $|\lambda|^{-1/\alpha}$, so

$$\frac{d \ln T_\lambda}{d \ln |\lambda|} = -\frac{1}{\alpha}. \tag{17}$$

A fault that shifts a slow eigenvalue by a given fraction changes that mode's time scale by twice that fraction at $\alpha = 0.5$, and by the same fraction at $\alpha = 1$.

In a chaotic reservoir ($g\tau_m > 1$) linear response breaks down entirely, and single-neuron clamping can be catastrophic [2]. This analysis assumes the input-driven, echo-state regime.

5 Information-theoretic consequences

In the linear, noise-free limit with a retrained readout, removing a neuron costs exactly one unit of memory capacity, whatever the value of α. Every α-dependence in the fault tolerance of the three measures of [1] therefore comes from noise, nonlinearity or truncation. In all three, the key quantity is the neuron's *unique variance*, $\mathrm{Var}(x_j \mid x_{-j})$: the part of its state that no other neuron predicts.

The results below assume the linear FOR with Gaussian input or a large-N Gaussian approximation, so that the measures of [1] (their Eqs. 8, 11 and 12) have closed forms. $C = \mathrm{Cov}(x)$ is the state covariance and σ_u^2 the input variance.

5.1 Memory capacity, readout retrained

Linear memory capacity at lag k is $MC_k = r_k^\top C^{-1} r_k / \sigma_u^2$, with $r_k = \mathrm{Cov}(x_t, u_{t-k})$. Dropping neuron j as a regressor costs exactly

$$\Delta MC_k^{\text{readout}} = \frac{\beta_{kj}^2}{(C^{-1})_{jj}\,\sigma_u^2} = \frac{\beta_{kj}^2\,\mathrm{Var}(x_j \mid x_{-j})}{\sigma_u^2}, \qquad \beta_k = C^{-1} r_k. \tag{18}$$

That is the neuron's readout weight for lag k squared, times its unique variance. Summing over lags, with $S = \sum_k r_k r_k^\top / \sigma_u^2$ the input-explained part of C,

$$\sum_k \Delta MC_k^{\text{readout}} = \frac{e_j^\top C^{-1} S\, C^{-1} e_j}{(C^{-1})_{jj}} \;\le\; 1, \qquad \text{with equality when } S = C. \tag{19}$$

$S = C$ holds for a noise-free linear reservoir driven by i.i.d. input. There the faulty $(N-1)$-neuron network also generically keeps $MC = N - 1$ (memory capacity is bounded by the number of units [10]), so the total loss including the propagated channel is exactly 1, independent of α. With state noise (for example fixed-point quantization, or the spike-count fluctuations dropped by the linearization), $S < C$ and the loss falls below 1 by an α-dependent amount.

Table 3 shows a numerical check on six random linear GL networks ($N = 10$, $g\tau_m = 0.7$, $\tau_m = 4$ steps, $L = 400$, state noise $\sigma = 10^{-3}$, 300 lags, one random neuron removed).

α	MC, intact	Loss, readout only	Loss, whole faulty network	Relative loss
1.0	6.32	0.18 ± 0.10	0.41 ± 0.38	6.4%
0.7	5.30	0.08 ± 0.04	0.24 ± 0.16	4.4%
0.5	4.78	0.05 ± 0.02	0.23 ± 0.16	4.8%

Table 3: Memory-capacity loss from removing one neuron, with a retrained readout (mean ± s.d. over six networks).

Without noise, one network lost 1.01, 0.95 and 0.79 units at $\alpha = 1$, 0.7 and 0.5. The low-α shortfall from 1 is memory held at lags beyond the 300-lag window, as predicted.

Two things stand out. First, the propagated channel (whole-network loss minus readout-only loss) costs 1.3 to 3.6 times as much as the readout channel, and the ratio grows as α falls, consistent with long memory spreading the fault through the network. Second, lower α loses less capacity in absolute terms but also starts with less, and the relative difference is within the spread. Six small networks are only indicative.

5.2 Output error, readout fixed

In the online setting the readout is not retrained, so what matters is the output error. After the transient, its power is

$$\mathbb{E}[e^2] = \int \left| w'^{\top} H'(\omega) B' - w^{\top} H(\omega) B \right|^2 S_u(\omega) \, \frac{d\omega}{2\pi} \; + \; \left(w'^{\top} \delta^* - w_j \bar{x}_j \right)^2 . \tag{20}$$

The second term is the bias from the mean shift and, at a fixed operating point, is α-independent for an untruncated buffer. During the transient it is replaced by $(w'^{\top}\delta(t) - w_j\bar{x}_j)^2$, which carries the power-law tail of the recovery dynamics.

5.3 Information transfer

By the chain rule of mutual information, the information transfer $T = I(U_{\text{past}}; X_t)$ of [1] splits cleanly into a direct and a propagated loss:

$$\Delta T = \underbrace{I(U_{\text{past}}; X_j \mid X_{-j})}_{\text{direct: unique information}} + \underbrace{I(U_{\text{past}}; X_{-j}) - I(U_{\text{past}}; X'_{-j})}_{\text{propagated}} . \tag{21}$$

In the Gaussian model the direct term has a closed form, using $C_\xi = \mathrm{Cov}(x \mid U)$, the noise covariance:

$$I(U_{\text{past}}; X_j \mid X_{-j}) = \tfrac{1}{2} \ln \frac{\mathrm{Var}(x_j \mid x_{-j})}{\mathrm{Var}(x_j \mid x_{-j}, U_{\text{past}})} = \tfrac{1}{2} \ln \frac{(C_\xi^{-1})_{jj}}{(C^{-1})_{jj}} . \tag{22}$$

The same unique variance governs both this term and the memory-capacity loss. The propagated term has no fixed sign: removing a neuron also removes the noise it injected into the others.

5.4 Active information storage

For a stationary Gaussian reservoir, the active information storage $\mathrm{AIS} = I(X_t; X_{t+1})$ [14] is

$$\mathrm{AIS} = \tfrac{1}{2} \ln \frac{\det C_0}{\det\left(C_0 - C_1 C_0^{-1} C_1^{\top}\right)}, \qquad C_1 = \mathrm{Cov}(X_{t+1}, X_t). \tag{23}$$

Two cautions apply. During the recovery transient the reservoir is not stationary, and a deterministic drift is itself predictable, so windowed AIS estimates will be inflated; subtract the mean trajectory (estimated across input realizations) first. And because dropout lowers the effective coupling, $g\sqrt{1-p}$, it weakens the recurrent dominance that [1] associate with high AIS. Cumulative dropout should therefore lower AIS and shift the reservoir toward the input-driven end of their trade-off. That is a hypothesis to test, not a result.

5.5 Redundancy and the correlation peak

Mastin et al. [1] report that neuron-to-neuron correlations peak in the transition region of α. Higher correlation means smaller unique variance, so the theory predicts that the *readout channel* of a single-neuron fault is cheapest near that transition. Strong correlation also implies strong coupling, which enlarges the *propagated channel*. Which effect wins is an open question, and measuring both channels separately across α would answer it directly.

6 Spiking-specific effects

The linear theory holds while every neuron stays in its operating regime. Thresholds add three effects it cannot capture: neurons switching on or off, cascades of such switches, and a deafened neuron that keeps firing.

6.1 Threshold crossings

Removing neuron j changes the mean input of each postsynaptic neuron i by $-W_{ij}r_j$. Whether i's response stays linear depends on where $\bar{I}_i$ sits on its noise-averaged f–I curve, so the criterion is stated in input units. For an active neuron, the mean free potential $\bar{V}_i$ lies above V_{th}, and the distance from it to threshold carries no information.

- **Near-silent neurons.** The relevant distance is from $\bar{I}_i$ to the rheobase. (For a subthreshold neuron this equals $(V_{\text{th}} - \bar{V}_i)/G_{\text{DC}}$.) If $|W_{ij}r_j|$ exceeds it, i can go silent after losing excitation, or start firing after losing inhibition.
- **Active neurons.** Linear response needs $|W_{ij}r_j|$ to be small compared with the input range over which the slope f_i' changes appreciably.

When either condition fails, i's gain γ_i jumps, which changes $\tilde{W}$ itself, not just the state.

6.2 Silencing cascades

Treat each switch as a branching event. Let m be the expected number of neurons that switch as a direct result of one switch; m depends on the degree, the weight distribution and the distribution of input distances to the rheobase. The expected total number of neurons affected by one fault is then

$$\mathbb{E}[\text{cascade size}] = \frac{1}{1-m} \quad (m < 1), \qquad m \geq 1 \;\Rightarrow\; \text{macroscopic failure.} \tag{24}$$

At $m \geq 1$ a single fault can propagate through the whole network. α enters in two ways. Mastin et al. [1] show that lower α raises the effective rheobase, so how α is normalized (bias or input gain) sets the distribution of distances to the rheobase and therefore m. And because a fractional neuron answers a step change in drive with a power-law approach, a cascade unfolds more slowly at low α: each generation of switches takes longer to complete.

6.3 The deafened neuron keeps firing

Under F-in, neuron j loses all input, but two things keep it active:

- **Its GL buffer still holds pre-fault potentials.** Reset writes V_{reset} into the buffer but leaves older samples in place, so j's potential relaxes toward its bias-driven equilibrium along the fractional step response. It goes on spiking for a while, in an echo of its pre-fault activity that fades as a power law, capped by L.
- **Its bias may exceed the rheobase.** If b alone can drive j to threshold, it settles into regular, input-independent firing. The fault then acts as a periodic drive on its downstream neurons, which is narrowband interference rather than silence, and the readout sees a periodic component.

So "deafening" a fractional neuron is neither instantaneous nor necessarily silent. Whether F-in or F-all is the better model of a real hardware failure depends on where in the circuit the failure sits.

6.4 Which neurons matter

Combining the linear results with the spiking picture gives a vulnerability ranking that can be computed from the golden run, before any fault is injected:

- **Direct channel, readout fixed:** $|w_j| \cdot \text{std}(x_j)$.
- **Direct channel, readout retrained:** $\beta_j^2 \cdot \text{Var}(x_j \mid x_{-j})$, from Eq. (18).

- **Propagated channel, mean shift:** $\|W_{:,j}\| \cdot r_j$, the lost mean drive of Eq. (14), weighted by the slow-mode components of A'.
- **Propagated channel, fluctuation:** $\|W_{:,j}\| \cdot \gamma_j \cdot \mathrm{std}(x_j)$, the lost input-carrying drive, weighted the same way.
- **Cascade risk:** the number of postsynaptic neurons whose mean input lies within $|W_{ij} r_j|$ of the rheobase.

Neurons far below threshold ($r_j = 0$, $\gamma_j = 0$) are harmless to remove unless the readout depends on them. A silent neuron near threshold is not covered by this: it sends nothing now, but a fault elsewhere can switch it on.

7 Testable predictions and analysis plan

The theory yields seven predictions that can be checked against golden-run versus faulty-run differences (Table 4). The HiL rig produces exactly the quantity the theory is written in, $\delta(t) = x'(t) - x(t)$, because it can replay the same input with and without the fault.

#	Prediction	How to test	What would falsify it
P1	After a permanent fault, $\|\delta(t) - \delta^*\|$ decays as $t^{-\alpha}$ up to about L steps, then exponentially	Log-log slope of $d\delta/dt$ (about $-1-\alpha$) over a window starting after $\|\lambda\|^{-1/\alpha}$ steps	Slope independent of α, or no break near L
P2	A transient fault decays as $t^{-1-\alpha}$	Same fit on a fault of a few steps' duration	As P1
P3	The 1% recovery time is capped by Eq. (13) (about $2.5L$ for $\tau_m = 10$ steps), and peaks at intermediate α for short buffers	Sweep α and L at the hardware's τ_m; compare with Eq. (13) and Figure 1	Recovery time well above Eq. (13), or a monotone curve in α at $L = 64$, $\tau_m = 10$
P4	At a fixed operating point, the mean shift δ^* is independent of α, apart from the effective leak ε_L	For each α, compare the measured δ^* with that α's own prediction $A'^{-1} q_j r_j$ (Eq. 14), using its measured r and γ, with and without the ε_L correction	Residual mismatch that varies with α after correction
P5	With a retrained readout, a noise-free linear FOR loses exactly 1 unit of MC per neuron, for every α	Linear GL simulation with long lag windows	Loss that differs from 1 beyond finite-sample bias
P6	Per-neuron damage ranks by the vulnerability scores of the spiking section	Spearman correlation between predicted score and measured loss, over all N single-neuron faults	No correlation
P7	Cumulative random dropout lowers the effective gain as $g\sqrt{1-p}$ and, with noise, memory capacity falls with it	Fit the leading eigenvalues of the faulty linearized network as p grows	Spectral radius that does not shrink with p

Table 4: Testable predictions.

7.1 Plan

1. **Linear GL network in software.** Confirm P1–P5 and P7 exactly. This is cheap, and every deviation in the later stages can be attributed to spiking or hardware rather than to

the theory itself.

2. **Spiking GL-FLIF network in software.** Repeat the same faults with the model and normalization of [1]. Measure where the linear predictions break, estimate the branching ratio m, and test P6. Separate the direct and propagated channels by also running F-out.
3. **Fixed-point model.** Quantize states and coefficients to the FPGA's formats. Quantization noise enters as state noise, which per Section 5 is exactly what makes the memory-capacity loss depend on α.
4. **HiL.** Inject F-all at a logged cycle, record all neuron states from both the golden and faulty runs, and compute $\delta(t)$, recovery times and the information measures. With the hardware L fixed, P3 says which α values give the slowest recovery.

Use the same estimators as [1] for MC, AIS and T so that results compare directly with their Figs. 2 and 3. Their paper does not say which estimator was used for AIS and T, so that needs checking with the authors or their code.

8 Limitations and open questions

The closed-form results are exact for the linear FOR and approximate for the spiking one. The main gaps, and what each one affects:

- **Linearization.** All closed forms assume small perturbations around a stationary asynchronous state. They fail for neurons near threshold, for cascades with m close to 1, and in the chaotic regime.
- **The effective L is not just the buffer length.** Mastin et al. [1] (Table 1) show that coefficient precision also truncates the history. With UQ0.16 coefficients the usable history is 699 steps at $\alpha = 0.5$ and 99 at $\alpha = 0.9$. Use whichever of buffer length and precision limit is smaller when computing ε_L.
- **Discrete-time stability.** Matignon's criterion is for continuous time. The explicit GL scheme of Algorithm 1 has its own step-size condition, which should be checked for the Δt and τ_m used on the FPGA.
- **Slow asymptotics at low α.** The power-law tails appear only after $|\lambda|^{-1/\alpha}$ steps, which can exceed L. At low α the truncated, pre-asymptotic regime may be the only one that is ever observed.
- **Gaussian information measures.** The closed forms for T and AIS assume Gaussian statistics. Spike-train measures need an estimator (binning, Kraskov-type, or a Gaussian approximation on filtered rates), and the estimator used in [1] is not stated.
- **Small numerical checks.** The memory-capacity table rests on six networks of ten neurons. It shows the direction of the effects, not their size.
- **Fault scope.** Only whole-neuron removal is treated. Faults in the GL coefficient memory change α for every neuron sharing it and need their own analysis; the same perturbation framework applies, with ΔA replaced by a change in the kernel $C_L(z)$.

The most useful open question for the thesis is the one from the redundancy discussion: across α, does the readout channel shrinking near the correlation peak outweigh the propagated channel growing there?

9 Theory versus implementation (SPIRES)

The simulations use SPIRES (txmastin/spires, `src/neurons/flif_gl.c` and `src/reservoir.c`). The formulas above stay as written; these notes record where the implementation differs from the model they assume.

- **Hard threshold, no noise.** Spiking is deterministic (`V >= V_th`). The smooth, noise-averaged f–I curve can come only from input fluctuations, and with steady input the rate locks to rational values. The spike probability as a function of free potential (an escape function) is therefore fitted to golden-run data, as in the regression of Section 1.2, rather than added to the simulator as noise.

- **Reset matches the model.** Reset overwrites only the current history sample and leaves older samples in place. The actual potential and the free potential therefore obey the same GL recursion with the same input, and their difference is driven only by what the resets remove. Exactly,

$$V_{j,n} = V^f_{j,n} - \sum_{m \le n} \eta_{n-m}\, \kappa_{j,m}, \qquad \eta(z) = \frac{1}{C_L(z) + \Delta t^\alpha z^{-1}/\tau_m},$$

  where $\kappa_{j,m}$ is the value the update would have produced at step m minus V_{reset} on steps where the neuron is reset or held, and zero otherwise. The reset kernel η is the membrane filter's impulse response, $\eta(z) = H_\lambda(z)/\Delta t^\alpha$ at $\lambda = -1/\tau_m$ in Eq. (7), so it has the same power-law tail as G.

- **Absolute refractory period.** For t_{ref} after a spike, V is held at V_{reset} and written to the history each step. The reset becomes a clamp of n_{ref} steps rather than a single pulse, the rate is capped at $1/(n_{\text{ref}}+1)$ per step, and γ_j falls at strong drive. The equation in Eq. (1) omits t_{ref}. Because the refractory clock accumulates dt by repeated addition, n_{ref} can come out one step short when t_{ref}/dt is not exactly representable (for example $dt = 0.1$ and $t_{\text{ref}} = 0.3$ give 2 steps, not 3); with $dt = 1$ and integer t_{ref} it is exact.

- **Readout state is the post-reset V.** `compute_output` reads the membrane potential after reset, not the free potential. Treat each reset, with its refractory clamp, as a single pulse of mean size $\bar{\kappa}_j$, and the spikes by Eq. (3). Then the readout state's fluctuation is $\tilde{x}_j$ passed through the filter $1 - \bar{\kappa}_j \gamma_j\, \eta(z)$, and its mean is $\bar{V}_j - \bar{\kappa}_j\, r_j\, G_{\text{DC}}/\Delta t^\alpha$, since $\eta(1) = G_{\text{DC}}/\Delta t^\alpha$. The direct-channel and memory-capacity formulas use the free potential; expect deviations from this filter in measured direct losses and MC.

- **Two time scales.** Each input sample is held for $1/dt$ micro-steps, and the readout samples V once per macro-step. Recovery dynamics, τ_m, t_{ref} and the GL buffer run in micro-steps; MC lags and readout errors are in macro-steps. The numbers in this document assume $dt = 1$, where the two coincide.

- **Buffer length.** The buffer has T_{mem}/dt slots and the history sum runs to $k = T_{\text{mem}}/dt - 1$, so $L = T_{\text{mem}}/dt - 1$. The truncation leak pulls toward 0, not V_{rest}, which the formula for $\bar{V}_j$ already includes. History before step 0 is effectively zero, so discard a washout period.

- **One-step spike delay** (confirmed). Neuron i receives $W s_{n-1}$, matching $I_{i,n}$ in Eq. (1) and the z^{-1} in Eq. (7). Each spike adds $W_{ij}\, dt^\alpha$ to V.

- **Shared GL coefficients.** All neurons share one coefficient array, so α is uniform and a coefficient-memory fault affects every neuron at once.

- **Assumed settings.** The analysis assumes `SYNAPSE_SIMPLE` (other synapse types add a filter $K_{\text{syn}}(z)$ to $\tilde{W}$) and `PLASTICITY_NONE` (plasticity would let the golden and faulty runs' weights drift apart).

References

[1] T. Mastin, N. Anderson, S. McNeal, M. Tubbin and C. Teuscher, "Fractional-order systems for neuromorphic computing: software and hardware opportunities and challenges," *npj Unconventional Computing*, vol. 3, art. 24, 2026. doi:10.1038/s44335-026-00070-8.

[2] P. Vincent-Lamarre, G. Lajoie and J.-P. Thivierge, "Driving reservoir models with oscillations: a solution to the extreme structural sensitivity of chaotic networks," *Journal of Computational Neuroscience*, vol. 41, pp. 305–322, 2016. doi:10.1007/s10827-016-0619-3.

[3] H. Sompolinsky, A. Crisanti and H. J. Sommers, "Chaos in random neural networks," *Physical Review Letters*, vol. 61, no. 3, p. 259, 1988.

[4] J. Kadmon and H. Sompolinsky, "Transition to chaos in random neuronal networks," *Physical Review X*, vol. 5, art. 041030, 2015. doi:10.1103/PhysRevX.5.041030.

[5] K. Rajan, L. F. Abbott and H. Sompolinsky, "Stimulus-dependent suppression of chaos in recurrent neural networks," *Physical Review E*, vol. 82, art. 011903, 2010. doi:10.1103/PhysRevE.82.011903.

[6] N. Bertschinger and T. Natschläger, "Real-time computation at the edge of chaos in recurrent neural networks," *Neural Computation*, vol. 16, no. 7, pp. 1413–1436, 2004. doi:10.1162/089976604323057443.

[7] D. Matignon, "Stability results for fractional differential equations with applications to control processing," in *Computational Engineering in Systems Applications (CESA '96)*, Lille, France, vol. 2, pp. 963–968, 1996.

[8] I. Podlubny, *Fractional Differential Equations*, Mathematics in Science and Engineering, vol. 198. San Diego: Academic Press, 1999.

[9] R. Garrappa, "Numerical evaluation of two and three parameter Mittag-Leffler functions," *SIAM Journal on Numerical Analysis*, vol. 53, no. 3, pp. 1350–1369, 2015. doi:10.1137/140971191.

[10] H. Jaeger, "Short term memory in echo state networks," GMD Report 152, German National Research Center for Information Technology, 2002.

[11] F. L. Bauer and C. T. Fike, "Norms and exclusion theorems," *Numerische Mathematik*, vol. 2, pp. 137–141, 1960. doi:10.1007/BF01386217.

[12] T. Tao and V. Vu (with an appendix by M. Krishnapur), "Random matrices: universality of ESDs and the circular law," *Annals of Probability*, vol. 38, no. 5, 2010. doi:10.1214/10-AOP534.

[13] J. Aljadeff, M. Stern and T. Sharpee, "Transition to chaos in random networks with cell-type-specific connectivity," *Physical Review Letters*, vol. 114, art. 088101, 2015. doi:10.1103/PhysRevLett.114.088101.

[14] J. T. Lizier, M. Prokopenko and A. Y. Zomaya, "Local measures of information storage in complex distributed computation," *Information Sciences*, vol. 208, pp. 39–54, 2012. doi:10.1016/j.ins.2012.04.016.